

Ω -CWY 的有限语法与展开规则

定义与证明资料 / 中文版

版本：2026-09-17，极限列标采用 $D[b+1]$ 。

这里给的是候选系统的规则。将形式列标解释成真正的序数，以及证明由外顶端生成的域良序，仍是后续数学任务。

1. 有限、封闭的自索引结构

一个有限表达式 G 是有限列序列 $C_0, \dots, C_j$ 。空序列为 0。列的位置地址从 0 开始编号。

列 C_j 中的一行是 (p, P) ，其中 $p < j$ 是父列地址， P 是轮廓：

$$P = (s ; (D_1, v_1), \dots, (D_t, v_t))$$

约束如下：

- $D_1 > \dots > D_t > 0$ ，每个 D_i 是同类的**有限、封闭子表达式**，不能包含外顶端。
- $s < v_1 < \dots < v_t$ 。
- $s \leq p$ ，各 v_i 要么 $\leq p$ ，要么等于当前列地址 j 。
- 没有变点时只记录常值 s 。

各 D_i 内部有自己的局部编号；外层复制时绝不修改 D_i 内的地址。自索引是有限语法树，不是指针循环。

形式上， P 是分段常值函数：在 $\xi \geq D_1$ 时为 s ，越过 D_i 向下时换成 v_i 。 $P(0)$ 记作 q ，称起源地址：有变点时 $q = v_t$ ，否则 $q = s$ 。

同一列的 p 严格递减， P 严格递增。这是 skyline 正规形。

2. 比较与 skyline

比较规则自底向上递归定义：

- 表达式按列字典序，真前缀较小。
- 列按行字典序；先比较行的父地址 p ，再比较 P 。
- P 先比较 s ，再按变点表字典序；变点先比较 D_i ，再比较 v_i 。真前缀较小。

比较 D_i 时递归比较其有限表达式。由于进入严格子表达式，比较算法终止；这不等于已证明该排序关系良序。

对任意有限行集合作 skyline：同一 p 只保留最大的 P ；按 p 从大到小扫描，只保留比已保留最后一个 P 更大的轮廓。

3. 原式显示

一般写法：

$$[p : (s | \{D_1\} : v_1, \{D_2\} : v_2, \dots)]$$

花括号内为完整嵌套表达式，不是名称占位符。多个行用分号隔开；空列写 $[]$ 。零表达式显示 $\emptyset$ 。

若所有 D_i 都是有限整数，则显示成普通非降轮廓词：先写 s ，再把 v_i 重复 $D_i - D_{i+1}$ 次，其中 $D_{t+1} = 0$ 。

例如变点 $(3, 1), (1, 2)$ 、根值 0 显示为 $(\emptyset, 1, 1, 2)$ 。不使用重复幂记法。

反向读取有限轮廓词时，压去多余的开头根值重复，再按值改变的位置恢复变点；纯常值词归一为一个数。

4. 第 b 块的轮廓降低

b 从 0 开始。考虑起源 $q=P(0)$ ，在它的起源列 C_q 中找 P 的下邻近似：按该列既定顺序扫描。

- $Q < P$ 时，更新最近的 Q，继续扫描。
- 首次遇到一行 (r, Q) 满足 $Q \geq P$ 时，停止扫描。此时 P 有最小非零变点 D，且 $s \leq r < q$ 。
- 若所有行都小于 P，则返回最后一个；起源列为空则返回“无轮廓”。

在停止处，定义

$\delta = D[0]$	当 D 的末列为空（后继式）
$\delta = D[b+1]$	当 D 的末列非空（极限式）

把 P 在 δ 以上的值都截到 r， δ 以下仍取 q：

$L(\xi) = q$	$\xi < \delta$
$\min(P(\xi), r)$	$\xi \geq \delta$

在有限表上实现为：将原来的各地址值替换成与 r 的较小者，去掉相邻相同值；若 δ 非零，再追加变点 (δ, q) 。D 的原起源平台消失或降为 r。

返回 L 与最近的严格较小轮廓中较大的一个；没有最近轮廓就返回 L。这个过程只查起源列，不在此处拼接父列尾巴。

5. 末列局部降低 $D_b(G)$

设 G 末列非空，其最后一行为 (c, P) ，末列地址为 j，起源为 q。

1. 保留末列除最后一行外的所有行。
2. 求上一节的起源下邻近似。如果存在，给它父地址 c，并将其地址 q 提升为 j，加入候选行。
3. 将父列 C_c 的各行加入候选，其中轮廓地址 c 提升为 j。
4. 对候选行取 skyline。

所得列记为 $D_b(G)$ 。地址提升只作用于轮廓的自然数值，不进入列标子表达式。

6. 基本列

$0[n]=0$ 。任何非零有限式 G 的 $G[0]$ 都删除末列。末列为空的式子按后继处理，其所有编号展开都等于删除末列。

末列非空时，仍记最后控制行的父地址为 c，末列位置为 j，令 $w=j-c$ 。

先输出原来的 $C_0, \dots, C_{j-1}$ ，然后对 $b=0, \dots, n-1$ 逐块追加：

```
shift_b(D_b(G)),  
shift_(b+1)(C_(c+1)), ..., shift_(b+1)(C_(j-1)).
```

其中 shift_h 将所有外层地址 $a \geq c$ 改成 $a+h \cdot w$ ， $a < c$ 保持不变；父地址和轮廓值都作此搬运，嵌套列标内部不搬运。

每块恰有 w 列。第 b 块不依赖最终 n，因此 $G[n]$ 是 $G[n+1]$ 的逐列前缀；正编号增长时为真前缀。后继式的各项相同，仍满足非严格前缀关系。

一次展开仅递归调用输入中严格子列标的展开，且每次只产生有限个块。因此**一步展开算法的终止**由有限语法结构递归保证，不需要先假设整个排序良序。实际程序另加资源预算。

原末列之前的列不变。局部降低及 skyline 后首个替代列比原末列小，是字典序下降的局部机制；完整域的闭包、共尾性、良序性不能仅靠上述前缀性质代替证明。

7. 外顶端与标准域

$S(D)$ 是两列种子：第一列空，第二列只有 $(\emptyset, (\emptyset; (D, 1)))$ 。要求 D 非零。

```
T0 = 1
T(n+1) = S(Tn)
Limit[n] = Tn
```

每次 S 增加恰一层子表达式嵌套。外顶端只是有限候选的种子族，不允许作为内部列标，也不宣称它枚举了全部结构合法的手写式。

关注的标准域定义为这些种子在有限次基本列/前继操作下的可达式。程序的原式解析只检查语法和局部地址正规性，不判定可达性。

$D[b+1]$ 约定不能悄悄换回 $D[b]$ ：后者会使 T_3 的所有基本列项低于 T_2 。前者已通过相邻种子越界的有限检查，但一般共尾性还没有证明。

8. 计数与山脉图

对每个前缀，保持它之前的列不变，反复用 D_0 降低末列，直到末列空；再计一次删除空列。这给出计数视图。它不是“不断取整个表达式的第 1 项”的暴力下降链。

这里 D_0 中的极限列标仍按本版规则取列标的第 1 项。计数算法有预算；遇到时间、步数或内核资源限制时，保留已完成的精确前缀，当前及后续各列以 $?$ 占位，不显示当前列运行中的未完成数值。

为识别小层级，令 B_k 为一块：先一空列（块首位置 a ），后接 k 个相同的行列 $[a:(a)]$ 。把指数非增的 $B_{(k_1)} \dots B_{(k_m)}$ 解释为 $\omega^{k_1 + \dots + k_m}$ ；各块地址取自己的实际外层位置。

这些规范块的字典序与该多项式序一致。末尾指数 $k > 0$ 时，第零项将其换成 $B_{(k-1)}$ ；正编号 n 将其换成 $n+1$ 个 $B_{(k-1)}$ 。末尾 $k=0$ 时删去 B_0 。这一小片段对展开封闭。

$WW = [][\emptyset:(\emptyset)][1:(\emptyset)]$ 的第 n 项为 $B_{(n+1)}$ ，对应这片段的 ω^ω 界。新版进一步识别整个规范的有限康托树形片段，见下一节；绘图本身不再以 ω^ω 为上限。

画图时为每条父关系画一条轨道；0 层标起源值，在每个变点层标该点及其上方的地址值；斜线指向父列底点。每个实际变点都画出，恒值区间不重复列点。多父边使用同列内不同轨道避免数字重叠。最底部另写计数序列，包括 $?$ ；红色寻址数字不替换。

9. ε_0 以下层标的常见写法

康托正规式 $\alpha = \omega^{\beta_1 + \dots + \beta_m}$ ，其中 $\beta_1 \geq \dots \geq \beta_m$ ，每个 β_i 又是有限康托正规式。把 α 编成有序根森林：每个单项 ω^{β_i} 是一个根，它的孩子依次表示 β_i 的各个单项，递归下去；叶子表示 1。

按先根顺序输出这些树的节点，每个节点一列：

- 每棵树的根在位置 a 输出空列。
- 其每个非根节点，只输出一行 $(p, (a))$ ， p 是它的树父节点地址，常值轮廓始终指向这棵**外层树的根 a** 。
- 每个孩子列表及最外层根列表都要求按所表示的单项非增排列。

这给出了所有 ε_0 以下有限康托正规式的显式图编码。显示识别器反向检查单行、常值根、先根父关系及每一层的非增条件；任一条件不成立，就保留原式，绝不直接将非正规森林按序数加法“合并”后猜一个值。

例如：

$[][0:(0)]$	$\rightarrow \omega$
$[][0:(0)][0:(0)]$	$\rightarrow \omega^2$
$[][0:(0)][1:(0)]$	$\rightarrow \omega^\omega$
$[][0:(0)][1:(0)][0:(0)]$	$\rightarrow \omega^{(\omega+1)}$

这个识别不依赖观察到的计数。其排序和展开可按森林结构核对：第一次父指针/根分隔的差异对应康托正规式中第一次指数差异；非增条件保证这确实是正规式比较。

展开也保留该片段。设 $\alpha = \gamma + \omega^\beta$ 是末项分解：

- $\beta=0$ 时，末根为空列，删除它得到 γ 。
- β 是后继时，末叶的父节点正是外层根；降低列为空，复制块恰为 $\omega^{(\beta-1)}$ 的整棵树。因此 $\alpha[n] = \gamma + \omega^{(\beta-1)} \cdot (n+1)$ 。
- β 是非零极限时，末叶父节点不是外层根；父列嫁接与分块复制完全位于指数树中，递归给出 $\alpha[n] = \gamma + \omega^{(\beta[n])}$ 。这也说明上例 $\omega^{(\omega+1)}$ 的展开是 $\omega^\omega \cdot (n+1)$ ，并不是仅因字符串像一个树就随意赋予它序数含义。一般 Ω -CWY 的良序性并不由这个小片段的识别得到。

绘图对**所有有限层标**按原记号比较排序。识别成功的康托片段显示通常的 ω 幂、和与系数；更大或未识别的原式直接显示原式。长标签换行，层间距随标签增高，但内容不截断；只保留明确的图大小/文本/时间资源预算。

10. 尚未得出的结论

本实现及其有限测试不证明：

- 标准域没有无穷降链；
- 每条候选基本列在整个标准域中共尾；
- wY 的全域嵌入或与旧 CWY2/ $\omega+1$ 系统的完整对应；
- 某个特定公理体系可证明本候选良序；
- 所有结构合法但不可达的手写式都具有预期序数含义。

旧 CWY2 有限层标片段的样例一致性，以及新增有限自索引规则的可计算性，都不能代替这些结论。

11. NER 文件、保护与交付范围

NER 展开器 显示名为 **Ω -CWY**；内部 ID `omega-cwy2-self-v1` 保留兼容，不是名称仍叫 Ω -CWY2。三个显示模式仅为“原式”“计数序列”“山脉图”。导入后可输入 T_0 、 $T_2[3]$ 、 $Top[3]$ 、 W 、 W_2 、 WW 或完整列表。

可读源码分为 `core.mjs`、`counts.mjs`、`views.mjs` 和 `entry.mjs`。执行 `node build.mjs` 可在本目录重建同一独立 JS；无需第三方 Node 包。

单次核心接口约 700ms，计数约 450ms/30,000 次局部降低；另有列数、嵌套深度、结构大小、文本和缓存保护。计数采用 `BigInt`，超过预算不表示超出某个固定整数位宽。图超限时显示明确错误，不画局部图；计数超限则按第 8 节保留已完成前缀，其余每列写问号。

本文件收录已有规则、前缀与有限结构递归论证，以及已写出的康托片段说明；本次没有为候选补做整体良序、共尾或与 wY 比较的证明，也没有 Lean 证书。有限实现检查见验收记录，不能替代未完成的证明。